EviRecall

Evidence-Aware Memory for Long-Horizon Multi-Agent Tasks

Natural Language Processing (CSC5051 / CSC6052 / MDS5110)

Project Proposal | Fall 2026 | 28 October 2026

Project repository: github.com/SatouJotaro/evirecall

Member	Name	Student ID	University email
A	Bijun Zeng	226020337	bijunzeng@link.cuhk.edu.cn
B	Chengyu Tang	226010320	chengyutang@link.cuhk.edu.cn
C	Xiaochen Bai	225040170	xiaochenbai@link.cuhk.edu.cn

1 Introduction

A multi-agent system may retrieve the right information and still make the wrong decision if that information is no longer valid. Consider a travel-planning task in which the user reduces the budget after an initial itinerary has been saved. A planning agent may recall the old itinerary, an execution agent may act on it, and a verifier may accept the result because the agents agree. The difficulty is that the stored plan still has a valid source, while the constraint that justified it has changed. This example motivates our focus on the relationship between memory, evidence, and later actions.

EviRecall will investigate this problem by extending Smurfs with evidence-aware shared memory. We plan to connect its planning, execution, answering, and verification roles to a common memory interface, then study what happens when earlier conclusions encounter new evidence. The proposed intervention will track which observations support a summary, identify conclusions affected by an authoritative update, and retrieve the underlying records before those conclusions are reused. Its value will depend on whether this additional checking improves decisions enough to justify its cost.

The study has three connected questions. First, does tiered memory improve recall and task completion compared with recent context and ordinary retrieval? Second, does checking evidence validity reduce the reuse of stale information across roles and help the system recover after an update? Third, how do these effects change as the evidence budget and number of concurrent tasks increase? We will examine these questions through existing memory evaluations, a small multi-session action study, and a common experiment runner. The initial scope involves no parameter training; it concentrates on memory behavior and its consequences within a fixed agent system.

2 Related work and selection rationale

Our choices reflect the needs of this study: access to role-level decisions, a memory interface that can be modified independently, and evaluations that distinguish recall from action. The following comparisons are based on papers and source review. They establish a practical starting point for the project; implementation feasibility and experimental scale will be checked in an initial pilot.

2.1 Choosing a multi-agent backbone

Smurfs [1] provides an explicit tool-planning workflow with decomposition, execution, answering, verification, and recovery. These boundaries are useful because an error can be traced from the observation that entered the system to the role that later reused it. Smurfs also connects the project to the instructor's research. Its limitation is that persistent shared memory and a long-horizon task environment still require integration. We accept that work because keeping the backbone fixed gives us a clearer comparison of memory behavior, although it makes the first prototype less immediate than using a more complete application framework.

Deep Agents and LangGraph [8] are credible engineering alternatives: subagents, file/context tools, and persistence could shorten the route to a working application. However, their built-in state and memory behavior would also need to be separated from our intervention. For a course study centered on evidence passing between roles, we favor the narrower Smurfs workflow. Magentic-One [11] offers broader web, file, and code roles, but adds environment setup that our first experiment does not need; the reviewed AutoGen repository is also archived. TwinMarket [12] and EasyMED [13] would introduce financial or medical evaluation questions. They remain useful examples of multi-agent design, while Smurfs keeps our task definition closer to the memory question.

2.2 Choosing the memory mechanism

TierMem [2] is relevant because it links compressed memory to raw evidence and supports returning to those records when summaries are insufficient. We plan to begin with its lightweight Jev implementation [3], which exposes a small Python interface without requiring us to train the paper's router. This reduces the number of components we must reproduce before testing an intervention. The trade-off is dependence on Jev's judgment service, whose access, latency, and cost need to be established. Results from this implementation will be identified as Jev results rather than presented as a reproduction of the paper's trained system.

Other methods help define both the baseline and the limits of our contribution. A-MEM [14] and Mem0 [15] offer approaches to organizing and updating memory, but implementing several full backends would reduce the time available for controlled error analysis. CUP-Mem [10] is a higher-priority comparison because state revision and propagation-aware updates directly overlap our proposed mechanism. Governed Shared Memory [7] likewise establishes that provenance and supersession are existing ideas; its current service code differs from the paper artifact. Our intended contribution is a controlled study of evidence validity at Smurfs role handoffs. Recent-history and BM25 baselines will show whether the additional memory machinery offers benefits over simpler, cheaper alternatives.

3 Datasets and task design

LongMemEval [4] and STALE [10] will be the primary memory evaluations because they address complementary parts of the problem. LongMemEval contains 500 questions spanning information extraction, cross-session reasoning, knowledge updates, temporal reasoning, and abstention. Its cleaned S release gives us an established setting for testing whether memory remains useful on ordinary questions. STALE contains 400 scenarios with three queries each, covering state resolution, premise resistance, and policy adaptation. It puts more emphasis on whether later evidence changes what the agent should believe or do. Using both should expose a possible trade-off: a mechanism that aggressively rejects old memory might handle updates better while losing useful information on unchanged histories.

Memory QA alone cannot establish that several agents use evidence correctly during a task. We therefore plan a limited action study using MemoryArena [5], whose interdependent sessions require earlier experience to inform later actions. The first candidate is its travel environment, with existing tools wrapped for Smurfs. This choice introduces practical uncertainty: the reviewed code is marked as a preview and requires an additional travel database. We will test a complete multi-session group before committing to a larger study. If the environment cannot be reproduced within the project budget, we will retain controlled cross-role episodes and state that end-to-end long-horizon action has not been evaluated.

The remaining resources have supporting roles. MAST [6] provides more than 1,600 annotated traces and a taxonomy of 14 failure modes, which can inform our error labels. Those traces are not replayable Smurfs tasks or a memory-training set. MemoryAgentBench [9] offers a fallback for incremental memory testing, while LoCoMo [17] would support a conversational cross-check with TierMem's evaluation setting. Adding either is less urgent than completing the two primary evaluations and the action pilot, since a broader dataset list would not by itself strengthen evidence about our central failure mechanism.

LongMemEval and STALE are two evaluation resources, not a training/test pair. We will reserve complete histories, scenarios, or task groups for development and hold out separate groups for final evaluation. LongMemEval S, M, and oracle describe context settings rather than data splits. STALE uses LongMemEval-derived distractors, and MemoryAgentBench also shares some sources, so overlap must be checked and dependence disclosed. Gold answers, evidence-location labels, conflict explanations, and environment scoring state will remain outside the agent input. Memory will persist across the intended sessions of a task, while stores will be isolated between unrelated groups and experimental conditions.

4 Proposed method and implementation

4.1 Evidence validity at role handoffs

The memory adapter will retain immutable observations and the source IDs behind each summary. It will also record who supplied the observation, when it entered the task, and whether a later update affects it. Before a role reuses a summary, the adapter will check for unresolved changes to its supporting evidence. When an explicit update comes from the user or an authorized environment observation, dependent conclusions will be marked for review and the relevant raw records retrieved within a fixed budget. The system will preserve the old record so that the change remains inspectable. A statement produced by another agent will not acquire authority merely because several roles repeat it.

This mechanism involves a substantive trade-off. Invalidating every related memory would be easy to implement, but could discard useful facts and trigger unnecessary retrieval. Checking too little could allow a stale conclusion to pass through several roles with a convincing citation. We will therefore distinguish the existence of a source from its current relevance and semantic support, and preserve uncertainty when the update is ambiguous. Paired update/no-update cases will test whether the intervention reacts to a meaningful change without unnecessarily disrupting an unchanged task. These cases form a separate stress suite; their scores will not be presented as official MemoryArena results. STALE will test how far the method extends to implicit conflicts without supplying gold update labels to the agent.

4.2 Integration and engineering choices

We will start with a direct Python adapter around Jev's `observe()`, `add_summary()`, and `retrieve()` operations, with `promote()` write-back invoked explicitly. Smurfs will remain responsible for generating the answer, and its verifier will be extended to inspect the returned evidence. A separate MCP or HTTP service could make the memory component independently deployable, but would add another execution boundary before the research behavior is established. Similarly, reusing Jev's SQLite/FTS5 raw store and file-based summaries avoids introducing a new vector database as an additional variable. Writes will be serialized within a task group; independent groups can run concurrently with separate stores.

For model access, a configurable API client is the initial choice because it allows us to study memory without first operating a local serving stack. The exact provider, model version, context limit, and spending cap will be fixed after checking tool-call reliability and pilot cost. The task/answer model will remain identical across the main conditions, while Jev's judgment service and any evaluation models will be recorded separately. Local serving remains an option if resources are available, but GPU scheduling is outside the initial claim. A Python runner will save versioned configurations, JSONL events, and CSV summaries so that all three members can execute the same protocol and recompute results from saved outputs without requiring a tracking service.

5 Experimental design

5.1 Comparisons and experimental scale

The memory-QA comparison will include recent history, BM25 retrieval over identical raw chunks, unmodified Jev, and Jev with our intervention. Keeping the answer model and input histories fixed will help separate the effect of memory from changes in model capability. We will also attempt a matched CUP-Mem comparison on STALE, since it addresses the closest alternative explanation for any improvement. If its pilot is unsuccessful, the missing comparison will be disclosed and the resulting claims narrowed. The action study will compare Smurfs without persistent memory, Smurfs with Jev, and Smurfs with the intervention under identical tools, observations, and task limits.

Ablations will disable validity filtering and raw-evidence recovery separately to identify what each contributes. Claims that multi-agent execution itself is beneficial would additionally require a budget-matched single-agent control. We will begin with 10 development units per primary dataset and 3 travel groups, then use measured costs to assess provisional targets of 100 held-out LongMemEval questions, 100 STALE scenarios, and 20 travel groups. Before the final runs, we will freeze data IDs, prompts, model/judge versions, write-back policy, and budgets. Where affordable, three repetitions will help estimate variability; uncertainty will be grouped by shared history, scenario, or task rather than treating related queries as independent observations.

5.2 Quality, evidence, and recovery

Official scorers will provide the main dataset results, supplemented by category-level analysis and human checks. Evidence analysis will distinguish a citation that resolves to a record, retrieval of the necessary evidence, and actual support for the decision. For update episodes, stale-use rate will measure stale-dependent decisions among eligible post-update decisions. Cross-role propagation will measure the fraction of update episodes in which another role reuses stale evidence, and recovery rate will measure the fraction that end with a corrected, successful outcome. The annotation rubric will specify these denominators and retain failures. Two reviewers will independently label a prespecified sample, with disagreements and representative successful and failed traces reported alongside aggregate scores.

5.3 Resource measurements

The resource study will vary a small set of evidence budgets and, separately, task concurrency at levels 1, 2, and 4 on a fixed subset. This provides a manageable systems question: whether extra checking remains useful when retrieval is constrained or several tasks compete for API access. We will hold tools and retry policy fixed, measure throughput and latency including queue time, and count role, summary, retrieval, judgment, and retry calls even when a run fails. Ingestion, execution, and evaluation costs will be reported separately; missing provider usage will be marked unknown. These measurements describe API-level behavior, and injected delays will be labeled as synthetic rather than interpreted as GPU utilization or hardware-scheduling evidence.

6 Collaboration, feasibility, and expected outcomes

The agreed allocation gives each member a component to implement and a question to investigate. Xiaochen Bai will focus on memory integration and the intervention, Bijun Zeng on task evidence and error recovery, and Chengyu Tang on the shared runner and resource evaluation. Each member will also run experiments, analyze results, and write the corresponding report sections. This division should allow independent progress once the interfaces are agreed, while keeping all three parts tied to the same experimental question.

Owner	Implementation and experiments	Report contribution
Xiaochen Bai	Memory baselines, Smurfs hooks, one update intervention, and mechanism ablations.	Method, integration, and ablation results.
Bijun Zeng	STALE input/label contract, one travel adapter, paired cases, and evidence/error analysis.	Task design, evidence analysis, and qualitative results.
Chengyu Tang	Common runner, QA loading, configs/splits, score wrappers, and usage/latency experiments.	Shared setup, efficiency, and reproducibility.

The main coordination task is to agree on task inputs, memory operations, evidence packets, trace events, and episode results before developing the full system. Xiaochen can then work against small fixtures, Bijun can score fixed traces, and Chengyu can run a dummy backend. An integrated example in the first week will expose interface problems early. Workload balance will come from limiting the scope to one intervention, one action environment, and one runner with two resource axes; if the pilot reveals an uneven burden, a bounded adapter or scorer can be reassigned. The benchmark owner will not be responsible for implementing the other members' methods or environments.

The first week is a feasibility stage for Smurfs/Jev access, STALE/CUP-Mem, the travel environment, and actual model costs. Weeks two and three will develop the agreed components and baselines using development data, followed by protocol freezing. Week four will run the main comparisons, ablations, and resource study; the final week will be used for cross-reproduction and report/poster preparation. These are internal stages rather than official course deadlines. If Jev or the action environment proves unavailable, the replacement or reduced scope will be documented explicitly so that the reported conclusions remain consistent with the implemented system.

The final report will be assembled around the shared protocol. Each member will draft their method and results sections, while the introduction, terminology, limitations, and conclusions will be edited together to explain how the findings relate. Deliverables will include the adapter and intervention, task cases, source-linked traces, reproducible configurations, and analysis, together with the course's LaTeX report, GitHub source, and poster presentation [16]. A useful outcome could be either improved recovery at an acceptable cost or evidence that the extra checking is too expensive or too aggressive for particular tasks; both would clarify when this form of memory control is worth using.

References

[1] Junzhi Chen, Juhao Liang, Benyou Wang. Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning. 2025. NAACL, pp. 3281-3298. [Paper] [Code]

[2] Qiming Zhu, Shunian Chen, Rui Yu et al.. From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. 2026. arXiv:2602.17913. [Paper] [Code]

[3] FreedomIntelligence. Jev TierMem: Lightweight Agent Memory. 2026. Software documentation; accessed 2026-10-27. [Code]

[4] Di Wu, Hongwei Wang, Wenhao Yu et al.. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2025. arXiv:2410.10813. [Paper] [Code] [Data]

[5] Zexue He, Yu Wang, Churan Zhi et al.. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. 2026. arXiv:2602.16313. [Paper] [Code] [Data]

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang et al.. Why Do Multi-Agent LLM Systems Fail?. 2025. arXiv:2503.13657. [Paper] [Code] [Data]

[7] Yanki Margalit, Nurit Cohen-Inger, Erni Avram et al.. Governed Shared Memory for Multi-Agent LLM Systems. 2026. arXiv:2606.24535. [Paper] [Code]

[8] LangChain. Deep Agents Overview. 2026. Official software documentation; accessed 2026-10-27. [Docs] [Code]

[9] Yuanzhe Hu, Yu Wang, Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. 2026. arXiv:2507.05257. [Paper] [Code] [Data]

[10] Hanxiang Chao, Yihan Bai, Rui Sheng et al.. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?. 2026. arXiv:2605.06527. [Paper] [Code] [Data]

[11] Adam Fourney, Gagan Bansal, Hussein Mozannar et al.. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. 2024. arXiv:2411.04468. [Paper] [Code]

[12] Yuzhe Yang, Yifei Zhang, Minghao Wu et al.. TwinMarket: A Scalable Behavioral and Social Simulation for Financial Markets. 2025. arXiv:2502.01506. [Paper] [Code]

[13] Bingquan Zhang, Xiaoxiao Liu, Yuchi Wang et al.. Human or LLM as Standardized Patients? A Comparative Study in Medical Education. 2025. arXiv:2511.14783. [Paper] [Code]

[14] Wujiang Xu, Zujie Liang, Kai Mei et al.. A-Mem: Agentic Memory for LLM Agents. 2025. arXiv:2502.12110. [Paper] [Code]

[15] Prateek Chhikara, Dev Khant, Saket Aryan et al.. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. 2025. arXiv:2504.19413. [Paper] [Code]

[16] NLP Course, CUHK-Shenzhen. CSC5051/CSC6052/MDS5110 Natural Language Processing, Fall 2026. 2026. Course website; accessed 2026-10-27. [Source]

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov et al.. Evaluating Very Long-Term Conversational Memory of LLM Agents. 2024. arXiv:2402.17753. [Paper] [Code]